

CI/CD Architecture, Secrets & Supply Chain Security

Enterprise deployment strategies, secrets management, SLSA, Sigstore, and supply chain h

TOPICS COVERED

- › CI / CD / CDeploy Distinctions
- › Canary Releases
- › Feature Flags & Progressive Delivery
- › Promotion Pipelines & Environment Gates
- › GitHub Secrets Architecture
- › OIDC Passwordless Cloud Auth
- › Dynamic Secrets & Rotation
- › Sigstore / Cosign / Rekor
- › Branch Protection & Rulesets
- › SBOM Generation
- › Blue-Green Deployments
- › Rolling Updates
- › GitOps Patterns
- › Quality Gates & Rollback
- › Organization & Environment Secrets
- › HashiCorp Vault Integration
- › Supply Chain Security (SLSA)
- › Artifact Attestations
- › Merge Queue
- › OpenSSF Scorecards

PART 11 — CI/CD Architecture

11.1 Continuous Integration, Delivery, and Deployment

Stage	Definition	Human gate?	Example
Continuous Integration (CI)	Automated build and test on every commit	No	Run tests on PR
Continuous Delivery (CD)	Software always releasable; deploy is a choice (release decision)	Yes	Staging auto-deploy, prod manual
Continuous Deployment (CDD)	Every passing commit deploys to production automatically	No	Tech startups, SaaS products

Enterprise Pipeline Architecture

```
# Multi-stage promotion pipeline:
name: Promotion Pipeline
on:
  push:
    branches: [main]

jobs:
  # Stage 1: Verify
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make test
      - run: make lint
      - run: make security-scan

  # Stage 2: Build artifact (once, promote the same binary)
  build:
    needs: ci
    outputs:
      image: ${ steps.build.outputs.image }
    steps:
      - id: build
        run: |
          IMAGE="ghcr.io/myorg/app:${ github.sha }"
          docker build -t $IMAGE .
          docker push $IMAGE
          echo "image=$IMAGE" >> $GITHUB_OUTPUT

  # Stage 3: Deploy to staging
  deploy-staging:
    needs: build
    environment:
      name: staging
      url: https://staging.example.com
    steps:
      - run: helm upgrade --install app ./charts/app
        --set image.tag=${ needs.build.outputs.image }
        --namespace staging

  # Stage 4: Integration tests on staging
  integration-tests:
    needs: deploy-staging
    steps:
      - run: pytest tests/integration/ --base-url=https://staging.example.com

  # Stage 5: Deploy to production (requires approval)
  deploy-production:
    needs: integration-tests
```

```
environment:
  name: production # Has required reviewers configured
  url: https://api.example.com
steps:
  - run: helm upgrade --install app ./charts/app
    --set image.tag=${{ needs.build.outputs.image }}
    --namespace production
```

11.2 Deployment Strategies

Blue-Green Deployment

Maintain two identical production environments (blue and green). Deploy to the inactive environment, run smoke tests, then switch traffic. Provides instant rollback by switching back.

```
# Blue-Green with AWS ALB:
- name: Deploy to Green
  run: |
    # Deploy new version to green target group
    aws ecs update-service \
      --cluster prod \
      --service api-green \
      --force-new-deployment

- name: Wait for green to stabilize
  run: |
    aws ecs wait services-stable \
      --cluster prod --services api-green

- name: Smoke test green
  run: curl -f https://green.internal.example.com/health

- name: Switch traffic to green (atomic)
  run: |
    aws elbv2 modify-rule \
      --rule-arn $RULE_ARN \
      --actions Type=forward,TargetGroupArn=$GREEN_TG_ARN

- name: Keep blue running for 30min rollback window
  run: sleep 1800 # In practice, use a separate workflow

# Rollback (instant):
- name: Switch back to blue
  if: failure()
  run: |
    aws elbv2 modify-rule \
      --rule-arn $RULE_ARN \
      --actions Type=forward,TargetGroupArn=$BLUE_TG_ARN
```

Canary Deployment

Gradually shift traffic to the new version, monitoring metrics at each step. Limits blast radius of issues to a small percentage of users before full rollout.

```
# Canary with Argo Rollouts (GitOps):
# rollout.yaml:
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: api-service
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 5 # 5% → canary
        - pause:
            duration: 10m # Wait and monitor
```

```

    - setWeight: 20    # 20%
    - pause:
      duration: 10m
    - setWeight: 50    # 50%
    - pause:
      duration: 10m
    - setWeight: 100   # Full rollout
analysis:
  templates:
    - templateName: success-rate-analysis
  args:
    - name: service-name
      value: api-service
---
# Analysis template with Prometheus metrics:
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate-analysis
spec:
  metrics:
    - name: success-rate
      interval: 5m
      successCondition: result[0] >= 0.95 # 95% success rate
      provider:
        prometheus:
          address: http://prometheus:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service-name}}",status!="5.."}[5m]))
            /
            sum(rate(http_requests_total{service="{{args.service-name}}"}[5m]))

```

GitOps Pattern

GitOps uses Git as the single source of truth for declarative infrastructure and application state. A GitOps operator (Flux or ArgoCD) continuously reconciles the cluster state with the desired state in Git.

```

# GitOps workflow:
# 1. Developer merges PR to application repo → CI builds image, pushes to GHCR
# 2. CI updates the image tag in the GitOps config repo
# 3. ArgoCD/Flux detects the config change and applies it to the cluster

# GitHub Actions step to update GitOps repo:
- name: Update image tag in GitOps repo
  run: |
    git clone https://github.com/myorg/k8s-config gitops
    cd gitops
    # Update the image tag using yq:
    yq e '.spec.template.spec.containers[0].image = "${{ env.IMAGE }}"' \
      -i apps/api-service/deployment.yaml
    git config user.email "ci@myorg.com"
    git config user.name "GitHub Actions"
    git add -A
    git commit -m "chore: update api-service to ${{ github.sha }}"
    git push
  env:
    IMAGE: ghcr.io/myorg/api-service:${{ github.sha }}
    GIT_TOKEN: ${ secrets.GITOPS_PAT }

```

11.3 Environment Protection Rules

```

# Environment configuration (via GitHub UI or API):
# Settings > Environments > production:
# - Required reviewers: [alice, bob] (at least 1 must approve)
# - Wait timer: 10 minutes (cooldown)
# - Deployment branches: main only
# - Deployment tags: v* (semantic version tags)
# - Prevent self-review: enabled
# - Custom environment secrets: DATABASE_URL, PROD_API_KEY

# Using environments in workflow:

```

```
deploy:
  environment:
    name: production
    url: https://api.example.com
  # This job will PAUSE until a required reviewer approves it
  steps:
    - run: echo "Deploying after approval..."

# Check if deployment was approved programmatically:
- name: Get deployment status
  run: |
    STATUS=$(gh api /repos/${{ github.repository }}/deployments \
      --jq '[0].environment')
    echo "Environment: $STATUS"
```

PART 12 — Secrets Management

12.1 GitHub Secrets Architecture

GitHub secrets are encrypted at rest using libsodium (NaCl) public-key cryptography. The public key for a repository is used to encrypt the secret value client-side (in the browser or via API) before it's transmitted. GitHub stores only the encrypted ciphertext and never has access to the plaintext.

Secret Scoping

Scope	Set by	Available to	Override?
Enterprise	Enterprise admin	All orgs/repos in enterprise	Org/repo can override
Organization	Org admin	All repos in org (or selected)	Repo can override
Repository	Repo admin	All workflows in repo	Environment overrides
Environment	Repo admin	Jobs targeting that environment	Most specific wins

```
# Secret naming rules:
# - Cannot start with GITHUB_ prefix (reserved)
# - Case-insensitive but convention is UPPER_SNAKE_CASE
# - Max 64KB per secret
# - Max 100 organization secrets
# - Max 100 repository secrets
# - Max 100 environment secrets

# Accessing secrets:
${{ secrets.MY_SECRET }}      # Direct access
${{ secrets.GITHUB_TOKEN }}   # Auto-injected token

# Secrets in matrix jobs – each matrix cell has access to same secrets
# Secrets are NOT available in pull_request events from forks
# (for security – use pull_request_target carefully instead)

# Set organization secret via API:
gh secret set DATABASE_URL \
  --org myorg \
  --visibility selected \
  --repos api-service,worker-service \
  --body "postgresql://..."

# Set environment secret:
gh secret set PROD_API_KEY \
  --env production \
  --repo myorg/api-service
```

Variables (Non-Secret Configuration)

GitHub Variables (distinct from Secrets) store non-sensitive configuration in plaintext. They are visible in logs and in the GitHub UI.

```
# Variables vs Secrets:
# Use secrets for: passwords, API keys, tokens, certificates
# Use variables for: URLs, region names, feature flags, non-sensitive config

# Set a variable:
gh variable set AWS_REGION --body "us-east-1" --repo myorg/api-service
gh variable set APP_VERSION --body "2.1.0" --org myorg

# Access in workflow:
```

```

${{ vars.AWS_REGION }}
${{ vars.APP_VERSION }}

```

12.2 OIDC — Passwordless Cloud Authentication

OIDC federation is the modern replacement for long-lived cloud credentials stored as GitHub secrets. See Part 3.7 for the technical flow.

```

# Enterprise OIDC setup guide:

# AWS — Create OIDC Identity Provider:
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1

# IAM Role trust policy (restrict to specific environment):
{
  "Statement": [{
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub":
          "repo:myorg/api-service:environment:production"
      }
    }
  }]
}

# Azure — Federated credential on App Registration:
az ad app federated-credential create \
  --id $APP_ID \
  --parameters '{
    "name": "github-production",
    "issuer": "https://token.actions.githubusercontent.com",
    "subject": "repo:myorg/api-service:environment:production",
    "audiences": ["api://AzureADTokenExchange"]
  }'

# GCP — Workload Identity Federation:
gcloud iam workload-identity-pools create github \
  --location=global --display-name="GitHub"
gcloud iam workload-identity-pools providers create-oidc github \
  --location=global --workload-identity-pool=github \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,
    attribute.repository=assertion.repository"

# Bind service account:
gcloud iam service-accounts add-iam-policy-binding \
  deploy@project.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/.../attribute.repository/myorg/api-service"

```

12.3 HashiCorp Vault Integration

For enterprises requiring dynamic secrets, audit trails, and centralized secret management, HashiCorp Vault integrates with GitHub Actions via the official vault-action.

```

# Vault integration workflow:
- uses: hashicorp/vault-action@v3
  id: vault
  with:
    url: https://vault.example.com
    method: jwt
# GitHub OIDC JWT

```

```
role: github-actions
jwtGithubAudience: https://vault.example.com
secrets: |
  secret/data/production/database password | DB_PASSWORD;
  secret/data/production/api key | API_KEY;
  aws/creds/deploy access_key | AWS_ACCESS_KEY;
  aws/creds/deploy secret_key | AWS_SECRET_KEY;

- name: Use secrets from Vault
  run: echo "DB connected" # ${ steps.vault.outputs.DB_PASSWORD }} is masked

# Vault AWS dynamic secrets – credentials that auto-expire:
# The aws/creds/deploy path generates:
# - access_key and secret_key with 15-minute TTL
# - Auto-revoked after TTL or on Vault token expiry
# - Full audit trail of who requested credentials and when
```

■ *Dynamic secrets from Vault are the gold standard for cloud credentials. They expire automatically, are unique per job, and leave a full audit trail in Vault's audit log.*

PART 13 — Supply Chain Security

13.1 SLSA — Supply Chain Levels for Software Artifacts

SLSA (pronounced 'salsa') is a security framework from Google that establishes levels of supply chain integrity guarantees for software artifacts. SLSA 1-4 represent increasing levels of hardening.

Level	Requirements	What it prevents
SLSA 1	Scripted build (not manual), basic provenance	Ad-hoc builds, no audit trail
SLSA 2	Version-controlled build config, authenticated provenance	Tampering with build scripts
SLSA 3	Hardened build platform, source verified, non-falsifiable provenance	Compromised build system
SLSA 4	Two-person review, hermetic build, strong provenance	Insider threats, supply chain attacks

```
# Generate SLSA provenance with GitHub Actions:
# GitHub Actions itself satisfies SLSA 3 for hosted runners.

name: Build with SLSA Provenance
on:
  push:
    tags: ['v*']

jobs:
  build:
    outputs:
      digests: ${ steps.hash.outputs.digests }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build artifact
        run: |
          go build -o app-linux-amd64 .
          sha256sum app-linux-amd64 > checksums.txt

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: binaries
          path: |
            app-linux-amd64
            checksums.txt

      - name: Generate digest
        id: hash
        run: |
          echo "digests=$(cat checksums.txt | base64 -w0)" >> $GITHUB_OUTPUT

# SLSA Provenance generation:
provenance:
  needs: build
  permissions:
    actions: read
    id-token: write
    contents: write
  uses: slsa-framework/slsa-github-generator/.github/workflows/generator_generic_slsa3.yml@v2.0.0
  with:
    base64-subjects: ${ steps.hash.outputs.digests }
    upload-assets: true
```

13.2 Sigstore / Cosign / Rekor

Sigstore is an open-source project providing free, transparent, and automated signing infrastructure. It consists of Cosign (signing tool), Rekor (transparency log), Fulcio (certificate authority), and Policy Controller (Kubernetes admission controller).

```
# Keyless signing with Cosign (OIDC-based, no key management):
# In GitHub Actions:
- uses: sigstore/cosign-installer@v3

# Sign a container image:
- name: Sign container image
  run: |
    cosign sign --yes \
      --oidc-issuer https://token.actions.githubusercontent.com \
      ghcr.io/myorg/api-service@${ steps.build.outputs.digest }
# Creates:
# 1. Certificate from Fulcio CA (with OIDC identity embedded)
# 2. Signature
# 3. Entry in Rekor transparency log
# Certificate expires in 10 minutes – signature is permanent in Rekor

# Verify (in deployment pipeline or admission controller):
- name: Verify image signature
  run: |
    cosign verify \
      --certificate-identity-regexp="github.com/myorg/api-service" \
      --certificate-oidc-issuer="https://token.actions.githubusercontent.com" \
      ghcr.io/myorg/api-service:latest | jq .

# Sign a binary/file:
cosign sign-blob --yes \
  --output-signature app.sig \
  --output-certificate app.pem \
  app-linux-amd64

# Attach and verify SBOM to image:
cosign attest --yes \
  --predicate sbom.spdx.json \
  --type spdxjson \
  ghcr.io/myorg/api-service:latest

cosign verify-attestation \
  --type spdxjson \
  --certificate-identity-regexp="github.com/myorg/api-service" \
  --certificate-oidc-issuer="https://token.actions.githubusercontent.com" \
  ghcr.io/myorg/api-service:latest
```

13.3 Branch Protection and Rulesets

Branch protection rules (and the newer Repository Rulesets, which support organization-wide policies) enforce code review, status check, and signing requirements before code can be merged or pushed.

```
# Branch protection via GitHub CLI:
gh api repos/myorg/api-service/branches/main/protection \
  --method PUT \
  --field required_status_checks='{"strict":true,"contexts":["ci/test","security/codeql"]}' \
  --field enforce_admins=true \
  --field required_pull_request_reviews='{"required_approving_review_count":2,"dismiss_stale_reviews":true,"r' \
  --field restrictions=null \
  --field required_conversation_resolution=true \
  --field required_linear_history=true \
  --field allow_force_pushes=false \
  --field allow_deletions=false

# CODEOWNERS file (.github/CODEOWNERS):
# Global owners (review required for all files):
* @platform-team
```

```
# Service-specific owners:
/services/auth/      @security-team @auth-team
/services/payments/  @payments-team @security-team @cto

# Infrastructure as Code:
/terraform/          @infra-team
/k8s/                @platform-team

# GitHub Actions workflows:
/.github/workflows/  @platform-team @security-team
```

Repository Rulesets (Organization-Wide)

```
# Rulesets can be applied to ALL repositories in an org:
# Settings > Rules > Rulesets

# Example ruleset (via API):
gh api orgs/myorg/rulesets \
  --method POST \
  --input - << 'EOF'
{
  "name": "enterprise-main-branch-policy",
  "target": "branch",
  "enforcement": "active",
  "conditions": {
    "ref_name": {
      "include": ["refs/heads/main", "refs/heads/release/**"],
      "exclude": []
    },
    "repository_name": {
      "include": ["~all"],
      "exclude": ["sandbox-*"]
    }
  },
  "rules": [
    {"type": "deletion"},
    {"type": "non_fast_forward"},
    {"type": "required_linear_history"},
    {"type": "pull_request", "parameters": {
      "required_approving_review_count": 2,
      "dismiss_stale_reviews_on_push": true,
      "require_code_owner_review": true
    }},
    {"type": "required_status_checks", "parameters": {
      "strict_required_status_checks_policy": true,
      "required_status_checks": [
        {"context": "ci/test", "integration_id": 0},
        {"context": "security/codeql", "integration_id": 0}
      ]
    }
  ]
}
EOF
```

13.4 Merge Queue

Merge Queue solves the 'pending head problem': when multiple PRs are approved but each was tested against an older main. The queue batches PRs, tests each merge combination, and merges only when the test passes.

```
# Enable merge queue on branch protection:
# Branch protection > Require merge queue

# GitHub Actions merge_group event:
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  merge_group:      # Trigger CI in the merge queue
    types: [checks_requested]
# The merge queue:
```

```
# 1. PR approved + all checks pass
# 2. Developer clicks "Merge when ready" (or auto-merge)
# 3. GitHub creates a merge group branch:
#   gh-readonly-queue/main/pr-123-abc123
# 4. Merges main into it + merges pr-123
# 5. Runs CI on this synthetic branch
# 6. If CI passes: fast-forward main to include pr-123
# 7. If CI fails: remove from queue, notify developer
```

13.5 OpenSSF Scorecard

```
# Generate and track OpenSSF Security Scorecard:
name: Scorecard
on:
  branch_protection_rule:
  schedule:
    - cron: '30 1 * * 6' # Weekly Saturday
  push:
    branches: [main]

jobs:
  analysis:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      id-token: write
      contents: read
      actions: read
    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false
      - uses: ossf/scorecard-action@v2.3.3
        with:
          results_file: results.sarif
          results_format: sarif
          publish_results: true
      - uses: actions/upload-artifact@v4
        with:
          name: SARIF file
          path: results.sarif
      - uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: results.sarif

# Scorecard checks include:
# - Branch-Protection - Signed-Releases
# - Code-Review - Token-Permissions
# - Dangerous-Workflow - Vulnerabilities
# - Dependency-Update-Tool (Dependabot)
# - Fuzzing - SAST (CodeQL)
# - License - Security-Policy
```

Interview Questions — CI/CD, Secrets & Security

Q: Explain blue-green vs canary deployments and when to use each.

A: Blue-green maintains two full environments and switches 100% of traffic instantly — ideal for low-latency cutover and immediate rollback, but requires double the infrastructure. Canary shifts traffic gradually (5% → 20% → 100%) enabling metric-driven rollout with limited blast radius — ideal for risky changes or large user bases. Use blue-green for strict compliance requirements (no mixed versions); use canary for normal feature releases.

Q: What is SLSA and what does it provide at each level?

A: SLSA (Supply Chain Levels for Software Artifacts) is a framework defining increasing levels of build integrity. Level 1: scripted builds with basic provenance. Level 2: version-controlled build config with authenticated provenance. Level 3: hardened build platform with non-falsifiable provenance (GitHub hosted runners qualify). Level 4: two-person review and hermetic (fully isolated) builds.

Q: How does Sigstore keyless signing work?

A: Instead of managing long-lived signing keys, Sigstore uses short-lived X.509 certificates issued by Fulcio CA based on OIDC identity. The workflow: (1) request OIDC token from GitHub, (2) Fulcio issues a 10-minute certificate embedding the OIDC identity, (3) Cosign signs the artifact with the ephemeral key, (4) the certificate, signature, and artifact hash are logged in Rekor (public transparency log). Verification checks Rekor rather than trusting a stored key.

Q: Why is pull_request_target dangerous and when is it necessary?

A: pull_request_target runs in the base branch context with full secrets access, designed for PRs from forks. It's dangerous because if you add an actions/checkout step, you execute untrusted fork code with secret access. It's necessary for use cases like auto-labeling or commenting on fork PRs (which need write permissions). Safe pattern: use pull_request_target only for actions that use the event payload (github.event.*) without checking out code.

Q: What is the difference between GitHub Secrets, Variables, and Environments?

A: Secrets store sensitive encrypted values (API keys, passwords) masked in logs. Variables store non-sensitive configuration in plaintext (regions, URLs). Environments group secrets/variables with protection rules (required reviewers, deployment branches) for specific deployment targets. Secrets and variables can be scoped to enterprise, org, repo, or environment. Environment-scoped secrets override repo-scoped ones with the same name.